

Dynamic Split | Screen

Two-player online co-op where both players see both views, in the style of It Takes Two. Each machine renders its own player on one half and the other player's live camera on the other half, and level volumes switch the screen between split and full view.

Unreal Engine 5.8

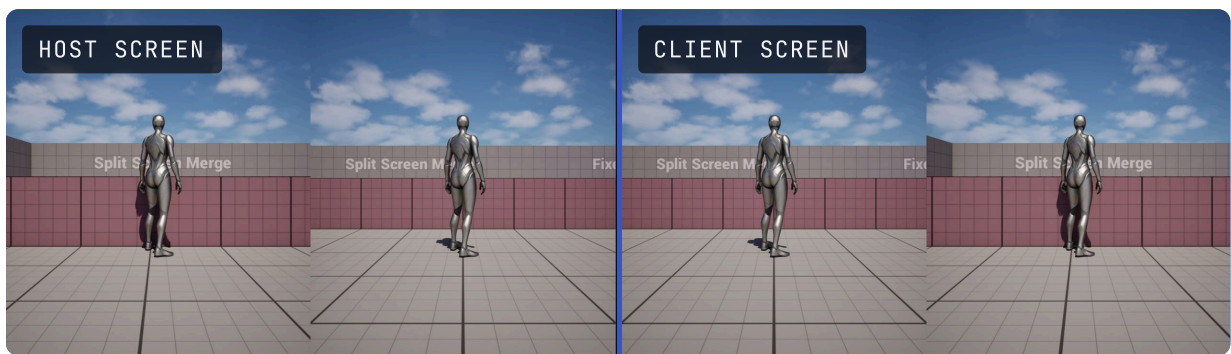
Win64

Listen server • 2 players

Network replicated

Runtime module: DynamicSplitScreen

Overview



The same moment on both machines. Each player sees their own character on the right and the other player's camera on the left.

- Split screen on **both** machines of an online session, not only on the host.
- No dummy LocalPlayer, PlayerController or SpectatorPawn. The second view is added straight into the renderer's ViewFamily, so HUD, audio, texture streaming and upscalers (DLSS / FSR / TSR) keep working.
- The other player's camera is replicated at 60 Hz and smoothed on the receiving side, including spring-arm collision.
- Five drop-in level actors: merge, fixed camera, follow camera, zoom, and a plain full-screen trigger.
- Smooth animated transition between split and full screen.

Installation

1. Install the plugin from Fab into your engine, or copy the `DynamicSplitScreen` folder into `<YourProject>/Plugins/`. A source copy is compiled the next time

you open the project.

2. Enable it in **Edit** › **Plugins** (search for *Dynamic Split Screen*) and restart the editor.
3. Set the viewport class in **Project Settings** › **Engine** › **General Settings** › **Default Classes** › **Game Viewport Client Class** to

`DynamicSplitScreenViewportClient`, or add this to

`Config/DefaultEngine.ini`:

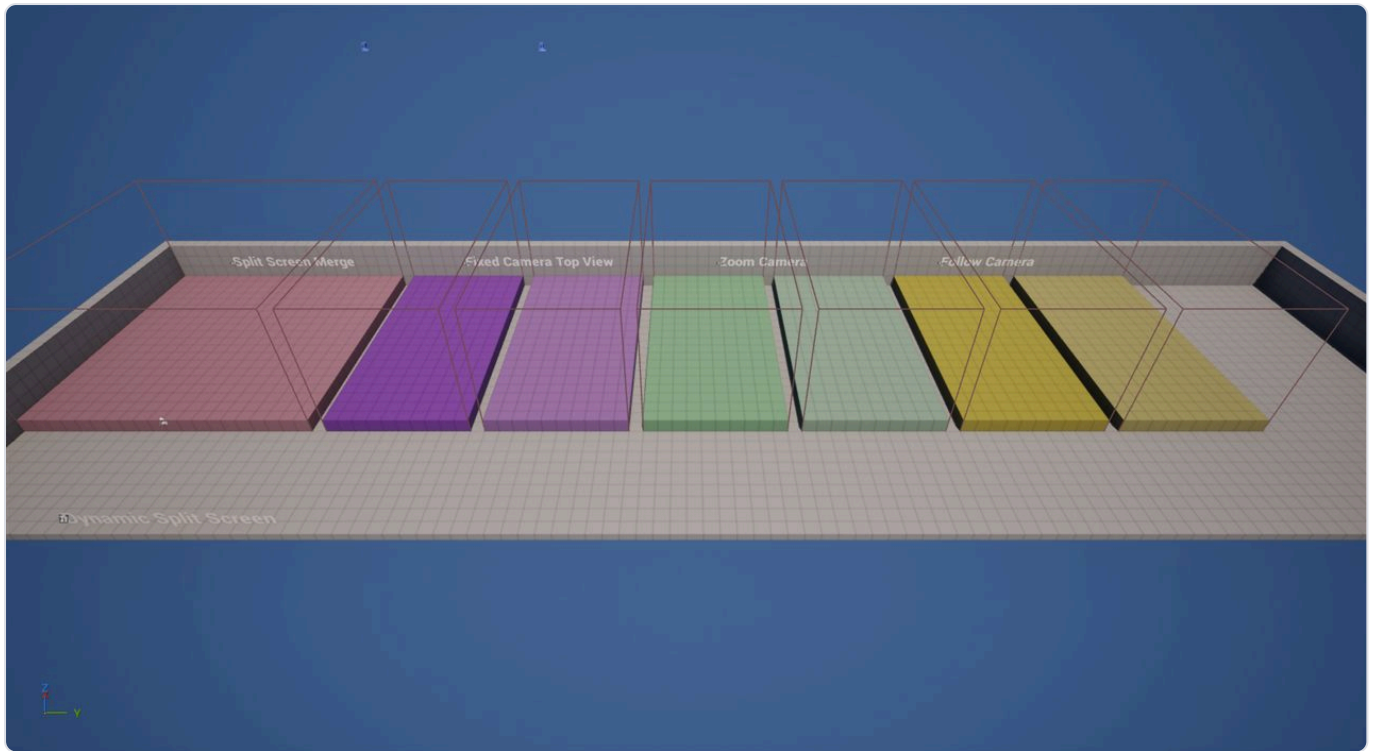
```
[/Script/Engine.Engine]
GameViewportClientClassName=/Script/DynamicSplitScreen.DynamicSplitScreenViewportClient
```

4. Make your GameMode inherit from `DynamicSplitScreenGameMode` and your PlayerController from `DynamicSplitScreenPlayerController`. Blueprint classes can simply pick these as parent classes.
5. Use a character with a `SpringArmComponent` and a `CameraComponent`, such as the standard Third Person template character. No other changes to the character are needed.

Dependencies: none beyond the engine. The plugin only uses the Core, CoreUObject, Engine, RenderCore, Slate and SlateCore modules.

Run the example project

1. Open `SplitScreen.uproject` with Unreal Engine 5.8. The demo map `/Game/DynamicSplitScreenDemo/Maps/DynamicSplitScreenMap` opens automatically.
2. Press **Play**. The project is preset to **Net Mode: Play As Listen Server** with **2 players**, so two windows open: *Server 0* (host) and *Client 1*.
3. Move with `W A S D`, look with the mouse, jump with `Space`. Click inside a window to control that player.
4. Walk onto the coloured pads. Each pad holds one of the volumes described below.



The demo map. From left to right: Split Screen Merge (pink), Fixed Camera (purple), Camera Zoom (green), Follow Camera (yellow). The lighter pad of each pair also switches the screen to full view.

If you press Play in **Standalone** mode there is only one player, so no split screen appears. The demo prints a reminder on screen and in the log in that case.

How it works

DynamicSplitScreenCameraProxy

One replicated actor per player. It carries that player's camera location, rotation, FOV and spring-arm length. The host fills its own proxy every frame; each client sends its camera to the server through an unreliable RPC.

DynamicSplitScreenSubsystem

Game instance subsystem. Every frame it finds the other player, reads their proxy, smooths the camera, sweeps it against walls, and runs split / full-screen transitions.

DynamicSplitScreenViewportClient

Replaces the game viewport's draw. It renders your own LocalPlayer's view plus a second scene view for the other player, side by side, in one ViewFamily.

`DynamicSplitScreenGameMode` spawns the proxies when players log in and enables split screen on the host once the second player joins.

`DynamicSplitScreenPlayerController` enables it on the client.

A transition only changes the screen of the machine that asked for it. When the host walks into a full-screen volume, the host's screen changes and the client keeps its split view, unless the volume is set up to react to both players (Split Screen Merge Volume).

Level volumes

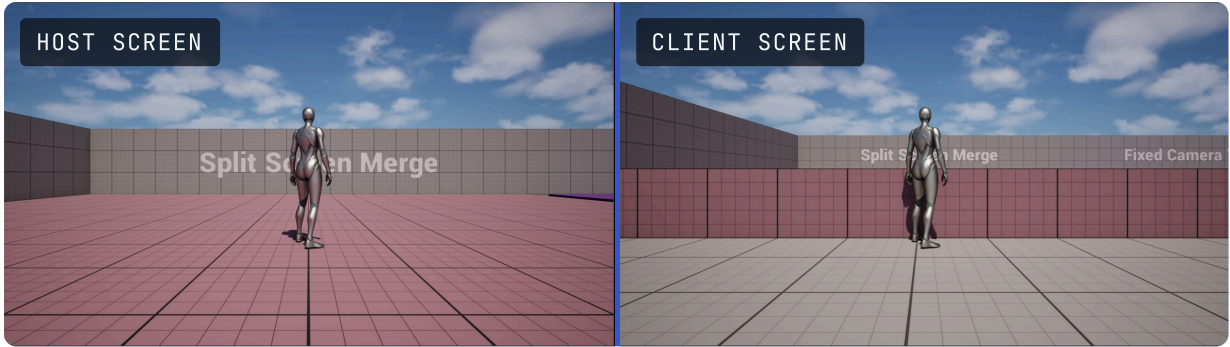
All volumes are C++ actors with ready-made Blueprints in `DynamicSplitScreenContent > Blueprints > Volumes`. Drag one into the level and scale its box.

Overlaps are counted per character, and full-screen requests are reference counted, so overlapping or neighbouring volumes hand over cleanly.

Split Screen Merge Volume

ASplitScreenMergeVolume · BP_SplitScreenMergeVolume

Merges the split screen into one full view when enough players (local or remote) are inside. Every machine switches to its own player's view. Use it for moments where the players are together.

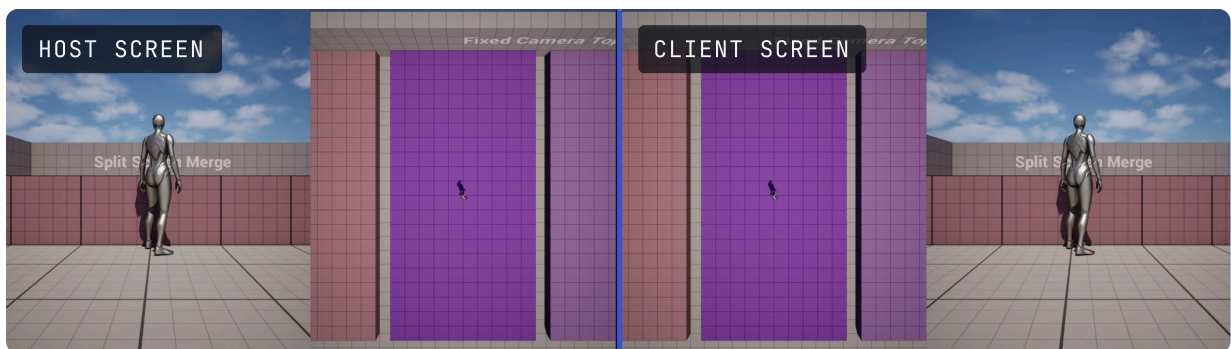


The host stands on the pink pad. The demo volume needs 1 player, so both screens merge.

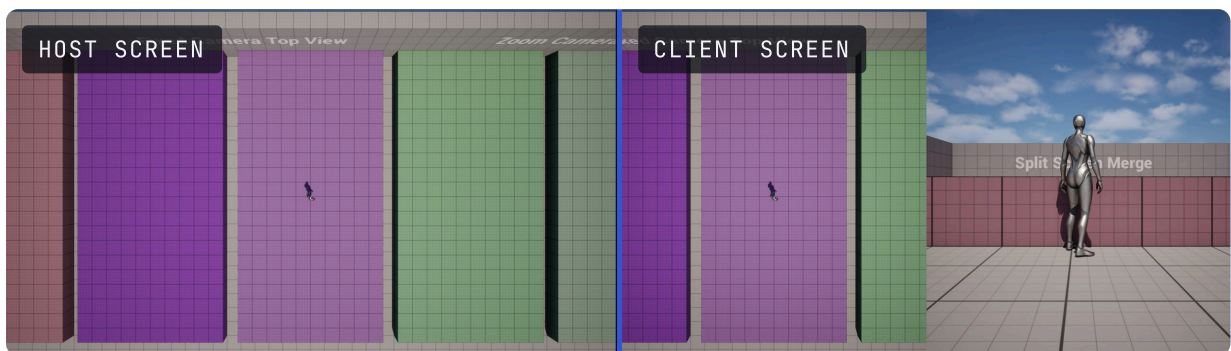
PROPERTY	DEFAULT	MEANING
PlayersRequiredToMerge	2	1 merges when anyone enters, 2 only when both players are inside.
bRestoreOnAnyPlayerExit	true	Restore the split as soon as the count drops below the requirement. If false, restore only when everyone has left.

■ Fixed Camera Volume `AFixedCameraVolume` · `BP_FixedCameraVolume`

Switches the entering player to the volume's own camera (position and angle set in the viewport). The control rotation blends to a fixed direction so movement stays intuitive, and look input is locked while inside. Optionally goes full screen.



Dark purple pad: split screen stays, the host's half becomes a top view. The client's left half shows the same fixed camera.

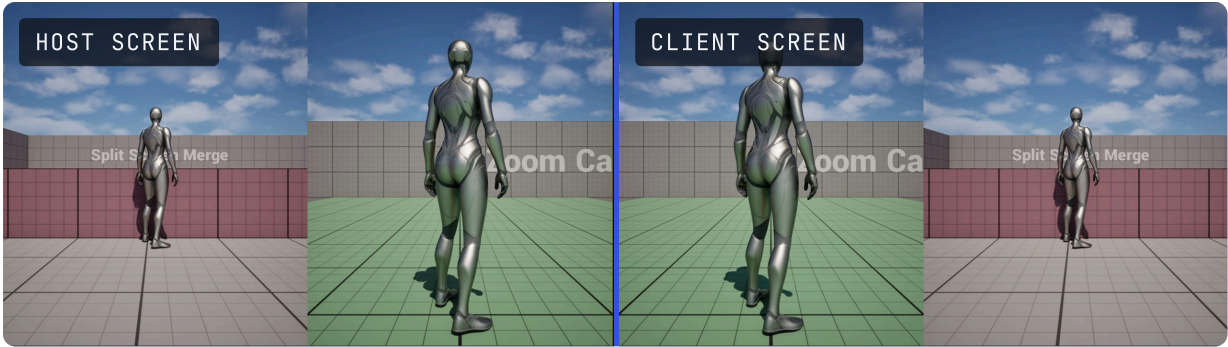


Light purple pad (`bUseSplitScreenTransition` on): the host goes full screen, the client keeps its split view.

PROPERTY	DEFAULT	MEANING
<code>FixedCamera</code>	<code>component</code>	Move and rotate this camera component to frame the shot.
<code>BlendTime</code>	<code>0.75 s</code>	Camera blend in and out.
<code>FixedControlRotation</code>	<code>(0, 0, 0)</code>	Control rotation used inside the volume, which defines what "forward" means.
<code>ControlRotationBlendTime</code>	<code>0.75 s</code>	How long the control rotation takes to turn.
<code>bRestoreControlRotationOnExit</code>	<code>false</code>	False keeps the current yaw on exit so movement does not jump; true restores the rotation from before entering.
<code>bUseSplitScreenTransition</code>	<code>false</code>	Also switch this screen to full view while inside.
<code>bFullScreenForEnteringPlayer</code>	<code>true</code>	React to the player this machine controls. If false, react to the local player with <code>FixedFullScreenPlayerIndex</code> .

Camera Zoom Volume `ACameraZoomVolume` · `BP_CameraZoomVolume`

Blends the entering player's spring-arm length to a new value and back on exit.
The other player's view of you follows the zoom, because the arm length is replicated.



Dark green pad: arm length 400 → 200, split kept.



Light green pad: zoom plus full screen for the host.

PROPERTY	DEFAULT	MEANING
TargetArmLength	200	Spring-arm length inside the volume. Smaller is closer.
ZoomInterpSpeed	5	Interp speed of the zoom (higher is snappier).
bUseSplitScreenTransition	false	Also switch this screen to full view while inside.
bFullScreenForEnteringPlayer	true	Same as on the Fixed Camera Volume.

Follow Camera Volume AFollowCameraVolume · BP_FollowCameraVolume

Spawns a camera that follows the character at a fixed offset and angle, for side-scroller or top-down sections. Look input can be locked while inside.



Dark yellow pad: the camera follows 30 m above the character, looking straight down.



Light yellow pad: follow camera plus full screen for the host.

PROPERTY	DEFAULT	MEANING
FollowOffset	(-500, 0, 300)	Offset from the character, in world space (or character space with <code>bFollowCharacterYaw</code>).
FixedCameraRotation	(-20, 0, 0)	Camera rotation. The demo uses pitch 270 for a straight-down view.
bFollowCharacterYaw	false	Rotate the offset with the character so the camera stays behind it.
BlendTime	0.75 s	Camera blend in and out.
bIgnoreLookInput	true	Lock mouse look while inside.
bUseSplitScreenTransition	false	Also switch this screen to full view while inside.

■ Split Screen Transition Trigger ASplitScreenTransitionTrigger

The simplest actor: the local player walks in and their screen goes full view; they walk out and it splits again. It changes no camera. It is not placed in the demo map; add it from **Place Actors** by searching for *Split Screen Transition Trigger*.

PROPERTY	DEFAULT	MEANING
bFullScreenForEnteringPlayer	true	React to the player this machine controls.
FixedFullScreenPlayerIndex	0	Local player index used when the option above is off.

Blueprint / C++ API

Everything is on the game instance subsystem. In Blueprint use **Get Game Instance Subsystem** with class *DynamicSplitScreenSubsystem*; in C++ call `GetGameInstance()→GetSubsystem<UDynamicSplitScreenSubsystem>()`.

Functions are in the *Dynamic Split Screen* category.

FUNCTION	WHAT IT DOES
EnableSplitScreen	Start drawing the other player's view. Called automatically by the GameMode (host) and PlayerController (client).
DisableSplitScreen	Back to a single view, for menus or cutscenes.
IsSplitScreenEnabled	True while split screen is active.
TransitionToFullScreen	Animate this machine's view to full screen.
TransitionToSplitScreen	Animate back to split screen.
IsInFullScreenMode	True once the full-screen transition has finished.
SetMainViewOnRight	True (default) puts your own view on the right, false on the left.
StartScreenFade	Fade both halves to black (1) or back (0) over a duration. Resets on map load.

From C++, volumes and other gameplay code should prefer

`RequestFullScreen(this)` / `ReleaseFullScreen(this)` : the screen stays full while any requester is active and returns to split when the last one releases or is destroyed.

Settings

Smoothing and collision of the other player's camera can be tuned in

`Config/DefaultGame.ini` :

```
[/Script/DynamicSplitScreen.DynamicSplitScreenSubsystem]
TransitionDuration=0.5
RotationInterpSpeed=45
bMainViewOnRight=True
```

SETTING	DEFAULT	MEANING
TransitionDuration	0.5 s	Split ↔ full-screen animation time.
RotationInterpSpeed	45	How fast the other player's camera rotation follows.
LocationInterpSpeed	25	Location smoothing while the other player uses a level camera (Fixed / Follow volumes).
AnchorInterpSpeed	60	Smoothing of the other character's position (absorbs network corrections).
ArmLengthInterpSpeed	30	Smoothing of the other player's zoom.
FOVInterpSpeed	12	Smoothing of the other player's FOV.
SnapDistance	1500 cm	Jumps larger than this snap instead of blending (teleport, respawn).
bDoCollisionTest	true	Sweep the other player's camera against walls, like a spring arm.
ProbeSize / ProbeChannel	12 / Camera	Sweep radius and collision channel.
bMainViewOnRight	true	Which side shows your own view.

The `GameMode` and `PlayerController` each have `bAutoEnableSplitScreen` (default true) if you want to start split screen yourself. The camera proxy's `SendRate` (default 60 Hz) sets how often clients send their camera.

Console: `DynamicSplitScreen.DebugDraw 1` draws the other player's reconstructed camera in the world.

Multiplayer testing

1. Open the Play options menu (the `:` next to Play) and set **Number of Players = 2** and **Net Mode = Play As Listen Server**. *Run Under One Process* can stay on. The example project already has these settings.
2. Choose **New Editor Window (PIE)** as the play mode so each player gets a window.
3. For a real network test, package the game, start one copy with `MyMap?listen` and connect the second with `open <host-ip>`.

The system is designed for exactly two players on a listen server. With a dedicated server there is no host camera to show, so split screen is not supported in that setup.

Troubleshooting

Only one view, no split.

You are playing Standalone, or with one player. Use Listen Server with 2 players (see above). The log shows `Split screen needs two players over the network` in this case.

Log warning: "Game Viewport Client Class is not DynamicSplitScreenViewportClient".

Set the viewport class as described in Installation, step 3, then restart the editor.

Split screen is active but the other half stays empty.

Your GameMode does not inherit from `DynamicSplitScreenGameMode`, so no camera proxies are spawned. Check the World Settings GameMode override of your map too.

The other player's camera clips through walls.

Make sure your level geometry blocks the `ProbeChannel` (Camera by default), or change the channel in the settings.

A volume changed my screen but not my partner's.

This is intended: transitions change the screen of the player who entered. Use the Split Screen Merge Volume when both screens should change together.

Dynamic Split Screen · Unreal Engine 5.8 · Screenshots captured from the example project running as a listen server with two players.